PROYECTO BASE DE DATOS

Tienda Informática

Alumnos	Joselu · Markel · Iván · Javier
Fecha	Abril 2026
Asignaturas	Bases de Datos · Entornos de Desarrollo
Sistema	tienda_informatica · MySQL 8 · JavaFX

SECCIÓN 1 — BASES DE DATOS	ER · Relacional · SQL · Datos iniciales
SECCIÓN 2 — ENTORNOS DE DESARROLLO	Clases · Casos de Uso · Secuencia · Actividades

SECCIÓN 1 — BASES DE DATOS

1.1 Diagrama Entidad-Relación

El modelo conceptual se desarrolló en iteraciones. Primero se identificaron las entidades principales, luego se añadió la especialización disjunta de clientes y finalmente se incorporó el atributo derivado *subtotal* y se ajustaron las cardinalidades.

Primera versión:

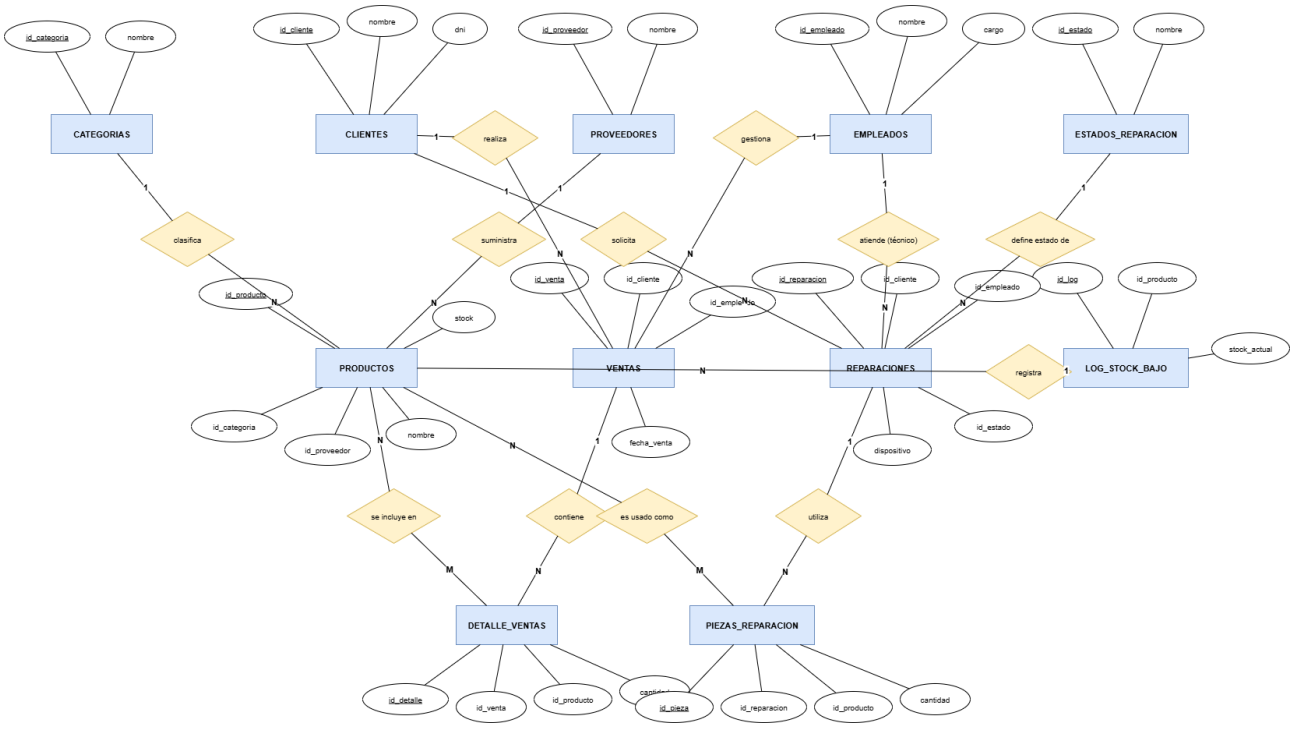

Figura 1a · ER — versión inicial

Versión final:

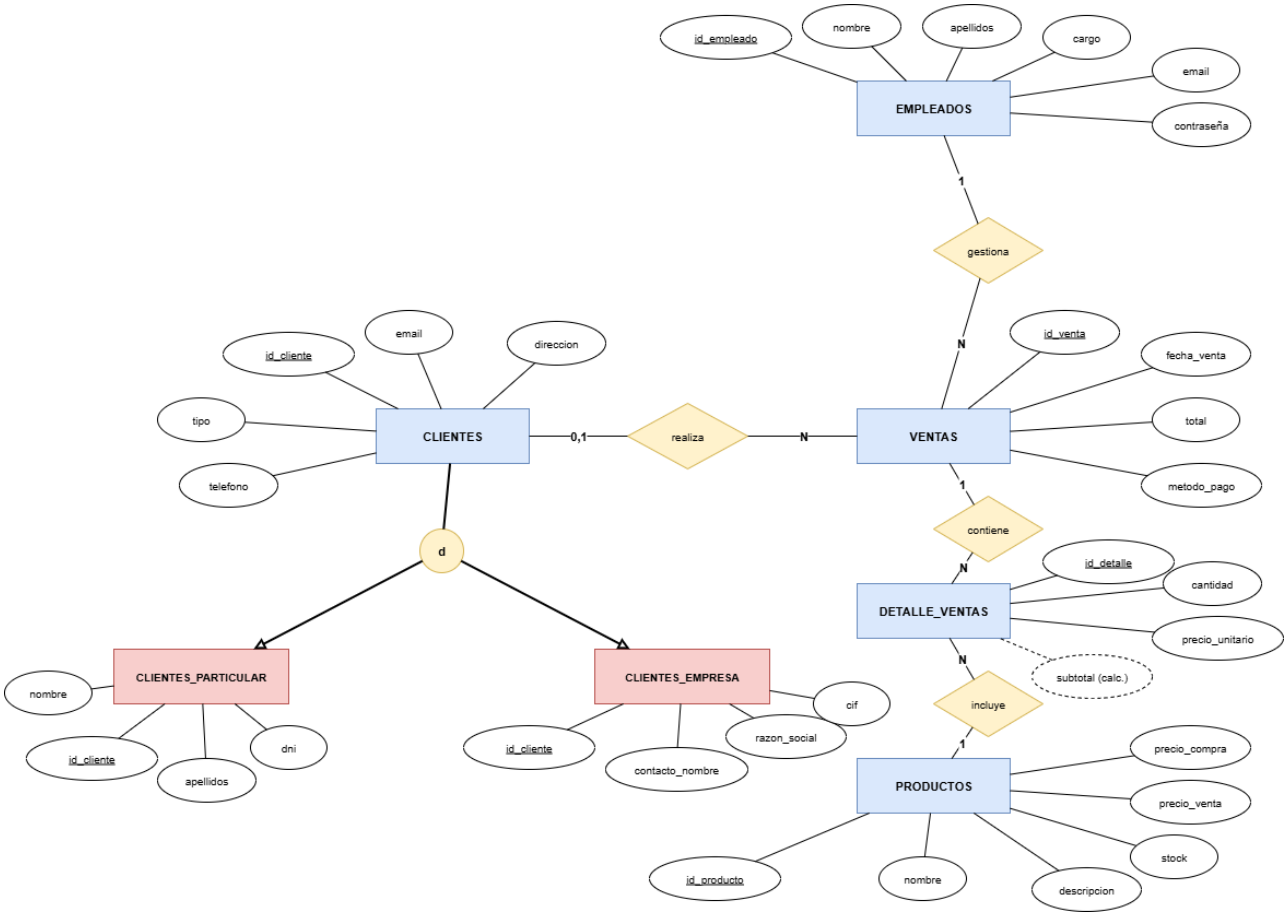

Figura 1b · ER final — versión entregable

1.2 Diagrama Relacional

El modelo relacional es la traducción directa del ER a tablas. Se aplicaron las reglas estándar: entidades fuertes → tablas, herencia disjunta → tabla-por-subtipo, atributo derivado → columna GENERATED ALWAYS.

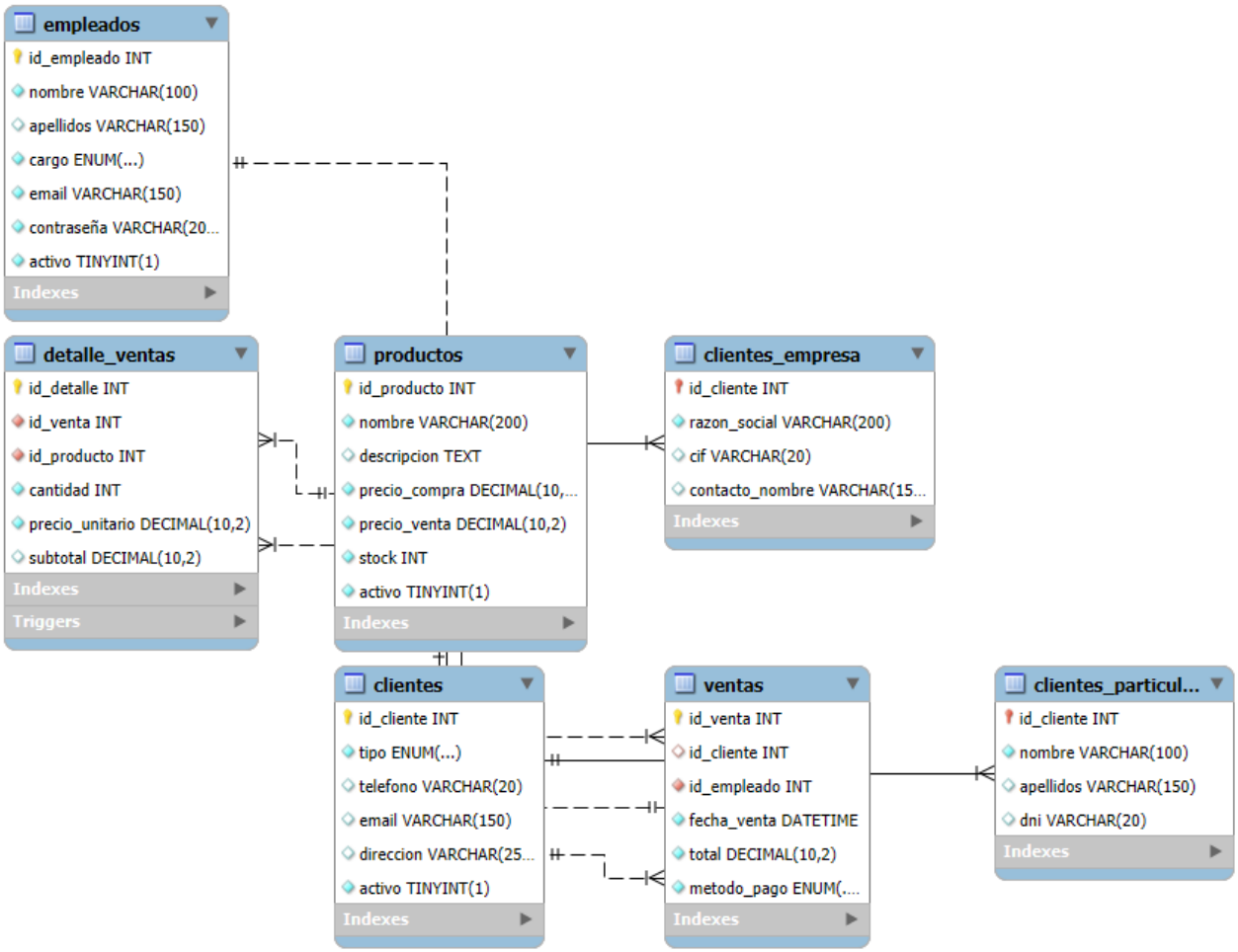

Figura 2 · Diagrama Relacional

Resumen de tablas:

Tabla	Clave primaria	Notas
empleados	id_empleado (AI)	ENUM cargo; email UNIQUE
clientes	id_cliente (AI)	Discriminador tipo ENUM
clientes_particular	id_cliente	FK→clientes CASCADE · dni UNIQUE
clientes_empresa	id_cliente	FK→clientes CASCADE · cif UNIQUE
productos	id_producto (AI)	stock INT NOT NULL
ventas	id_venta (AI)	FK→clientes SET NULL · total DECIMAL
detalle_ventas	id_detalle (AI)	FK→ventas CASCADE · subtotal GENERATED

1.3 Código SQL — Creación de la base de datos

El DDL implementa la herencia mediante el patrón tabla-por-subtipo. Se incluyen 4 triggers para integridad de stock y totales, y 2 usuarios MySQL con principio de mínimo privilegio.

Base de datos y productos

```
CREATE DATABASE IF NOT EXISTS tienda_informatica
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE tienda_informatica;

CREATE TABLE IF NOT EXISTS productos (
  id_producto INT NOT NULL AUTO_INCREMENT,
  nombre VARCHAR(200) NOT NULL,
  precio_compra DECIMAL(10,2) NOT NULL DEFAULT 0.00,
  precio_venta  DECIMAL(10,2) NOT NULL DEFAULT 0.00,
  stock INT NOT NULL DEFAULT 0,
  PRIMARY KEY (id_producto)
);
```

Empleados y herencia de clientes

```
CREATE TABLE IF NOT EXISTS empleados (
  id_empleado INT NOT NULL AUTO_INCREMENT,
  cargo ENUM('vendedor','tecnico','admin','gerente') NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  PRIMARY KEY (id_empleado)
);
-- SUPERCLASE
CREATE TABLE IF NOT EXISTS clientes (
  id_cliente INT NOT NULL AUTO_INCREMENT,
  tipo ENUM('particular','empresa') NOT NULL,
  PRIMARY KEY (id_cliente)
);
-- SUBCLASE 1
CREATE TABLE IF NOT EXISTS clientes_particular (
  id_cliente INT NOT NULL,
  nombre VARCHAR(100) NOT NULL, dni VARCHAR(20) UNIQUE,
  PRIMARY KEY (id_cliente),
  FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente) ON DELETE CASCADE
);
-- SUBCLASE 2
CREATE TABLE IF NOT EXISTS clientes_empresa (
  id_cliente INT NOT NULL,
  razon_social VARCHAR(200) NOT NULL, cif VARCHAR(20) UNIQUE,
  PRIMARY KEY (id_cliente),
  FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente) ON DELETE CASCADE
);
```

Ventas y detalle (atributo GENERATED)

```
CREATE TABLE IF NOT EXISTS ventas (
  id_venta INT NOT NULL AUTO_INCREMENT,
  id_cliente INT, id_empleado INT NOT NULL,
  fecha_venta DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  total DECIMAL(10,2) NOT NULL DEFAULT 0.00,
  metodo_pago ENUM('efectivo','tarjeta','transferencia') NOT NULL,
  PRIMARY KEY (id_venta),
  FOREIGN KEY (id_cliente)  REFERENCES clientes(id_cliente)  ON DELETE SET NULL,
  FOREIGN KEY (id_empleado) REFERENCES empleados(id_empleado)
);
CREATE TABLE IF NOT EXISTS detalle_ventas (
  id_detalle INT NOT NULL AUTO_INCREMENT,
  id_venta INT NOT NULL, id_producto INT NOT NULL,
  cantidad INT NOT NULL DEFAULT 1,
  precio_unitario DECIMAL(10,2) NOT NULL,
  subtotal DECIMAL(10,2) GENERATED ALWAYS AS (cantidad * precio_unitario) STORED,
  PRIMARY KEY (id_detalle),
  FOREIGN KEY (id_venta)     REFERENCES ventas(id_venta)     ON DELETE CASCADE,
```

```
  FOREIGN KEY (id_producto)  REFERENCES productos(id_producto)
);
```

Triggers de stock y total

```
DELIMITER $$
CREATE TRIGGER trg_stock_al_vender
BEFORE INSERT ON detalle_ventas FOR EACH ROW
BEGIN
  DECLARE v_stock INT;
  SELECT stock INTO v_stock FROM productos WHERE id_producto = NEW.id_producto;
  IF v_stock < NEW.cantidad THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Stock insuficiente';
  END IF;
  UPDATE productos SET stock = stock - NEW.cantidad
  WHERE id_producto = NEW.id_producto;
END$$

CREATE TRIGGER trg_actualizar_total_insert
AFTER INSERT ON detalle_ventas FOR EACH ROW
BEGIN
  UPDATE ventas SET total =
    (SELECT SUM(subtotal) FROM detalle_ventas WHERE id_venta = NEW.id_venta)
  WHERE id_venta = NEW.id_venta;
END$$
DELIMITER ;
```

Usuarios MySQL — mínimo privilegio

```
CREATE USER IF NOT EXISTS 'tienda_admin'@'localhost' IDENTIFIED BY 'admin_pass';
GRANT ALL PRIVILEGES ON tienda_informatica.* TO 'tienda_admin'@'localhost';

CREATE USER IF NOT EXISTS 'tienda_app'@'localhost' IDENTIFIED BY 'app_pass';
GRANT SELECT, INSERT, UPDATE ON tienda_informatica.* TO 'tienda_app'@'localhost';
FLUSH PRIVILEGES;
```

1.4 Datos iniciales de prueba

Se insertan datos que cubren todos los escenarios: empleados de distintos roles, cliente particular, cliente empresa, productos variados y dos ventas completas para verificar los triggers de stock y total.

Productos

```
INSERT INTO productos (nombre, precio_compra, precio_venta, stock) VALUES
  ('Portatil Lenovo IdeaPad', 450.00, 699.00, 10),
  ('Raton inalambrico Logitech', 15.00, 29.99, 50),
  ('Teclado mecanico RGB', 40.00, 79.99, 30),
  ('Monitor 24" Full HD', 150.00, 249.00, 15),
  ('SSD 1TB Samsung', 60.00, 109.99, 25);
```

Empleados

```
INSERT INTO empleados (nombre, apellidos, cargo, email, contrasena) VALUES
  ('Admin', '', 'admin', 'admin@tienda.com', 'admin1234'),
  ('Jose Luis', 'Perez', 'tecnico', 'joseluisperez@tienda.local', '12345'),
  ('Javier', 'Maza', 'vendedor', 'javiermaza@tienda.local', '12345'),
  ('Ivan', 'Lopez', 'gerente', 'ivan@tienda.local', '12345');
```

Clientes y venta de prueba

```
-- Cliente particular
INSERT INTO clientes (tipo, telefono, email) VALUES ('particular', '612345678', 'ana@gmail.com');
INSERT INTO clientes_particular (id_cliente, nombre, dni)
  VALUES (LAST_INSERT_ID(), 'Ana Garcia', '12345678A');

-- Venta de prueba (triggers actualizan stock y total automáticamente)
INSERT INTO ventas (id_cliente, id_empleado, metodo_pago) VALUES (1, 3, 'tarjeta');
INSERT INTO detalle_ventas (id_venta, id_producto, cantidad, precio_unitario) VALUES
  (1, 2, 2, 29.99),   -- 2x Ratón → subtotal 59.98
  (1, 3, 1, 79.99),   -- 1x Teclado → subtotal 79.99
  (1, 5, 1, 109.99);  -- 1x SSD → subtotal 109.99
                      -- total venta = 249.96 (calculado por trigger)
```

SECCIÓN 2 — ENTORNOS DE DESARROLLO

2.1 Diagrama de Casos de Uso

Los casos de uso describen las interacciones entre los actores (Vendedor, Técnico, Admin, Sistema) y el sistema. Se identificaron en dos fases: primero los casos principales de venta, luego los de gestión administrativa.

Boceto inicial:

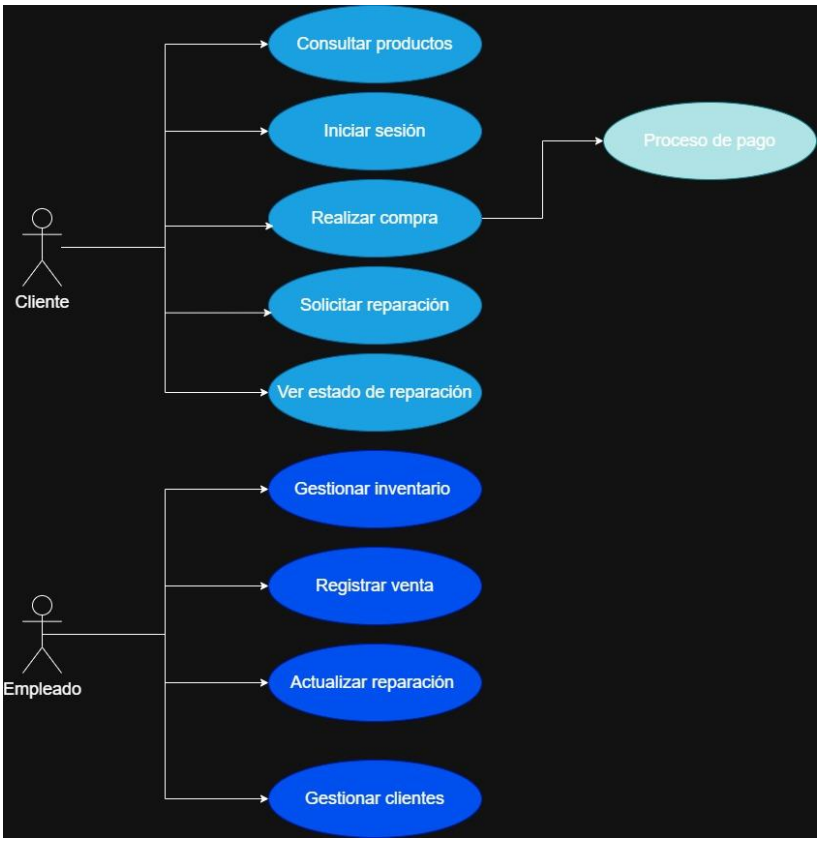

Figura 4a · Casos de Uso — boceto inicial

Versión final:

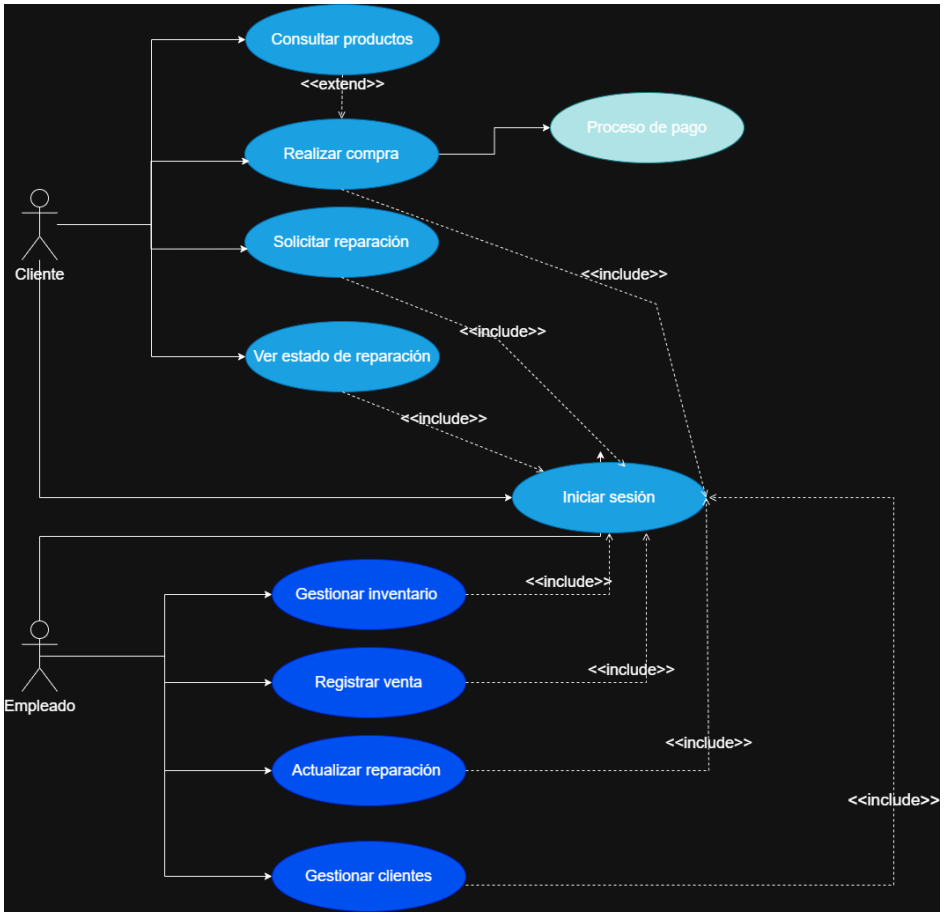

Figura 4b · Casos de Uso — versión final

2.2 Diagrama de Clases UML

El diagrama de clases modela la capa Java/JavaFX. Incluye la jerarquía de clientes con clase abstracta, relaciones UML (asociación, composición, herencia) y métodos de negocio como *calcularTotal()* en Venta y *getSubtotal()* en DetalleVenta.

Boceto inicial:

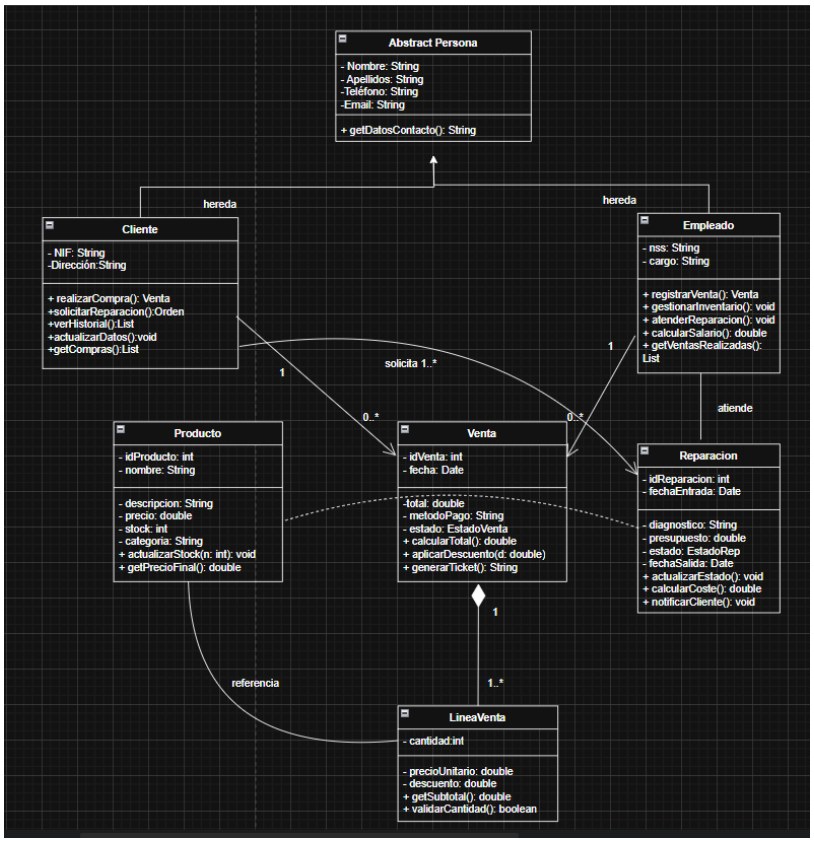

Figura 3a · Diagrama de Clases — boceto inicial

Versión final:

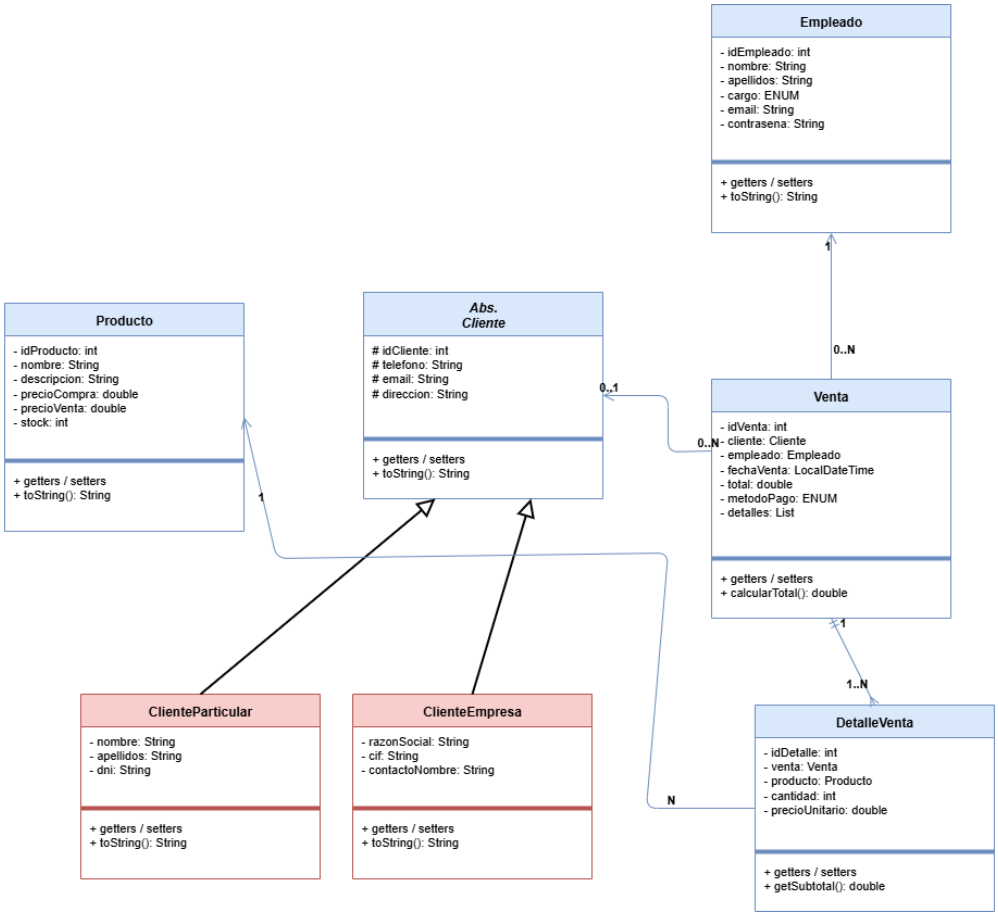

Figura 3b · Diagrama de Clases — versión final

2.3 Diagramas de Secuencia

Los diagramas de secuencia muestran la interacción temporal entre objetos para los tres flujos más representativos: registrar venta (flujo normal), error de stock insuficiente y registrar cliente particular.

2.3 Diagramas de Secuencia — Casos de uso seleccionados

Los diagramas de secuencia muestran la interacción temporal entre objetos para casos de uso específicos. Se elaboraron para los tres flujos más representativos: registrar venta, registrar cliente y error de stock.

Secuencia 1 · Registrar Venta (flujo normal)

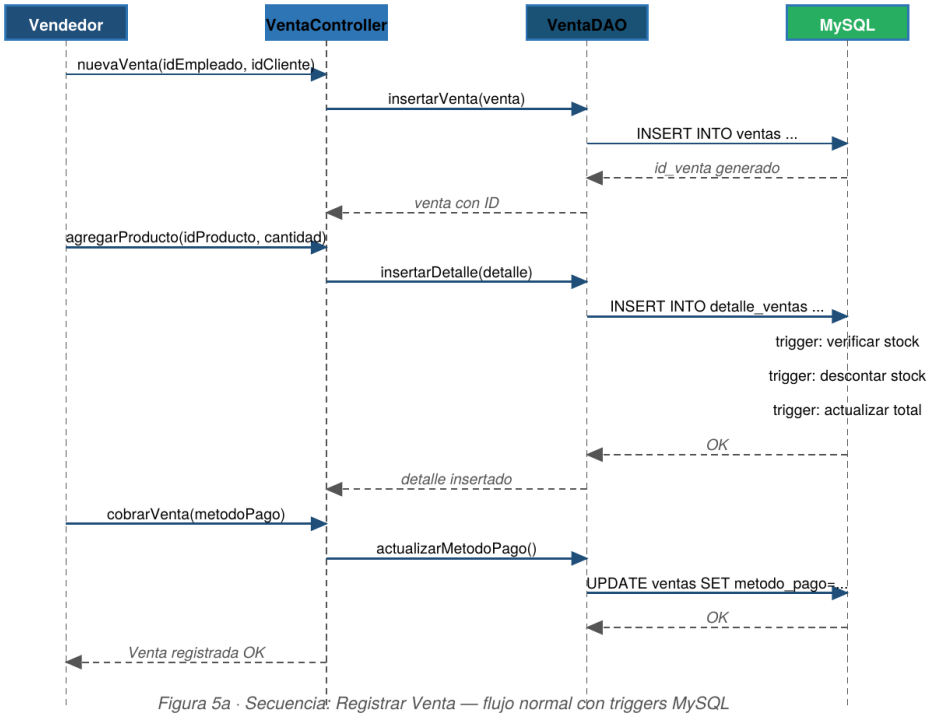

Figura 5a · Secuencia: Registrar Venta — flujo normal con triggers MySQL

Secuencia 2 · Error de Stock insuficiente

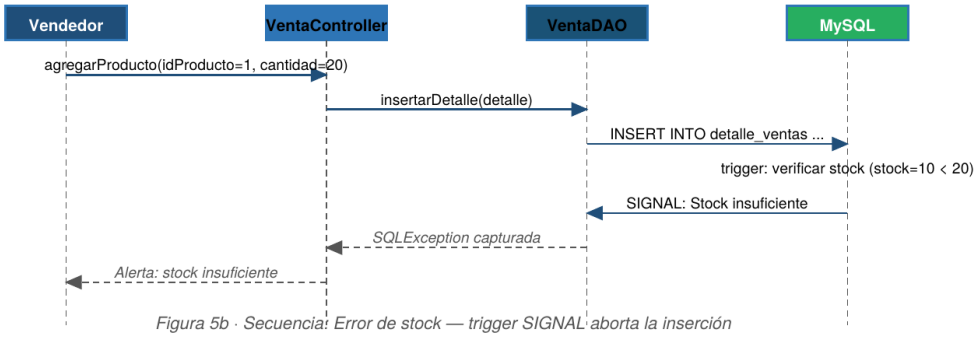

Figura 5b · Secuencia: Error de stock — trigger SIGNAL aborta la inserción

Figura 5a/5b · Secuencia: Registrar Venta y Error de Stock

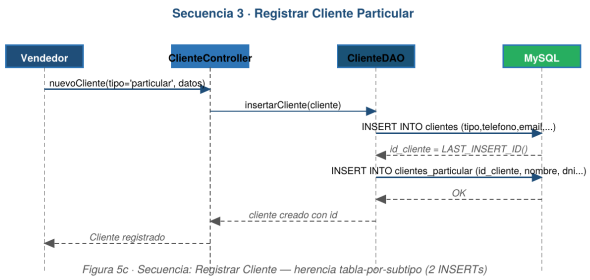

Figura 5c · Secuencia: Registrar Cliente Particular

2.4 Diagramas de Colaboración

Los diagramas de colaboración muestran los mismos flujos que los de secuencia pero enfatizando las relaciones estructurales entre objetos y el orden de los mensajes.

2.4 Diagramas de Colaboración — Relación entre objetos

Los diagramas de colaboración muestran los mismos flujos que los de secuencia pero enfatizando las relaciones estructurales entre objetos y el orden de los mensajes. Se representan para dos casos de uso.

Colaboración 1 · Registrar Venta

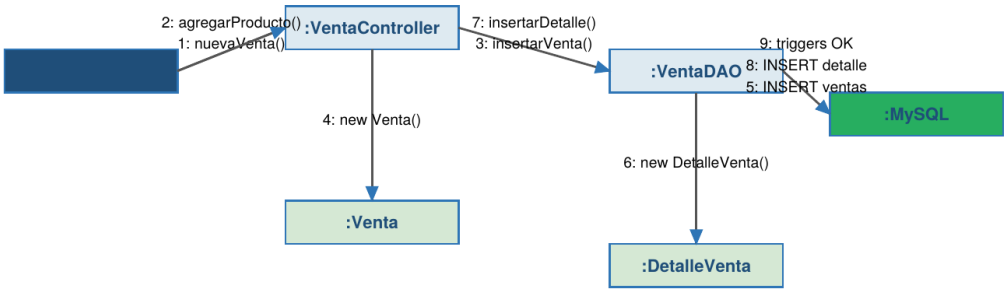

Figura 6a · Colaboración: Registrar Venta — mensajes numerados entre objetos

Colaboración 2 · Registrar Cliente (herencia)

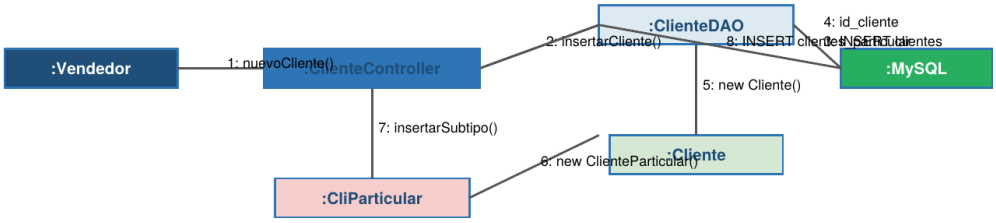

Figura 6b · Colaboración: Registrar Cliente — doble INSERT por herencia tabla-subtipo

Figura 6a/6b · Colaboración: Registrar Venta y Registrar Cliente

2.5 Diagramas de Actividades

Los diagramas de actividades modelan el flujo completo de las operaciones más importantes, incluyendo decisiones, condiciones de error y swimlanes por actor.

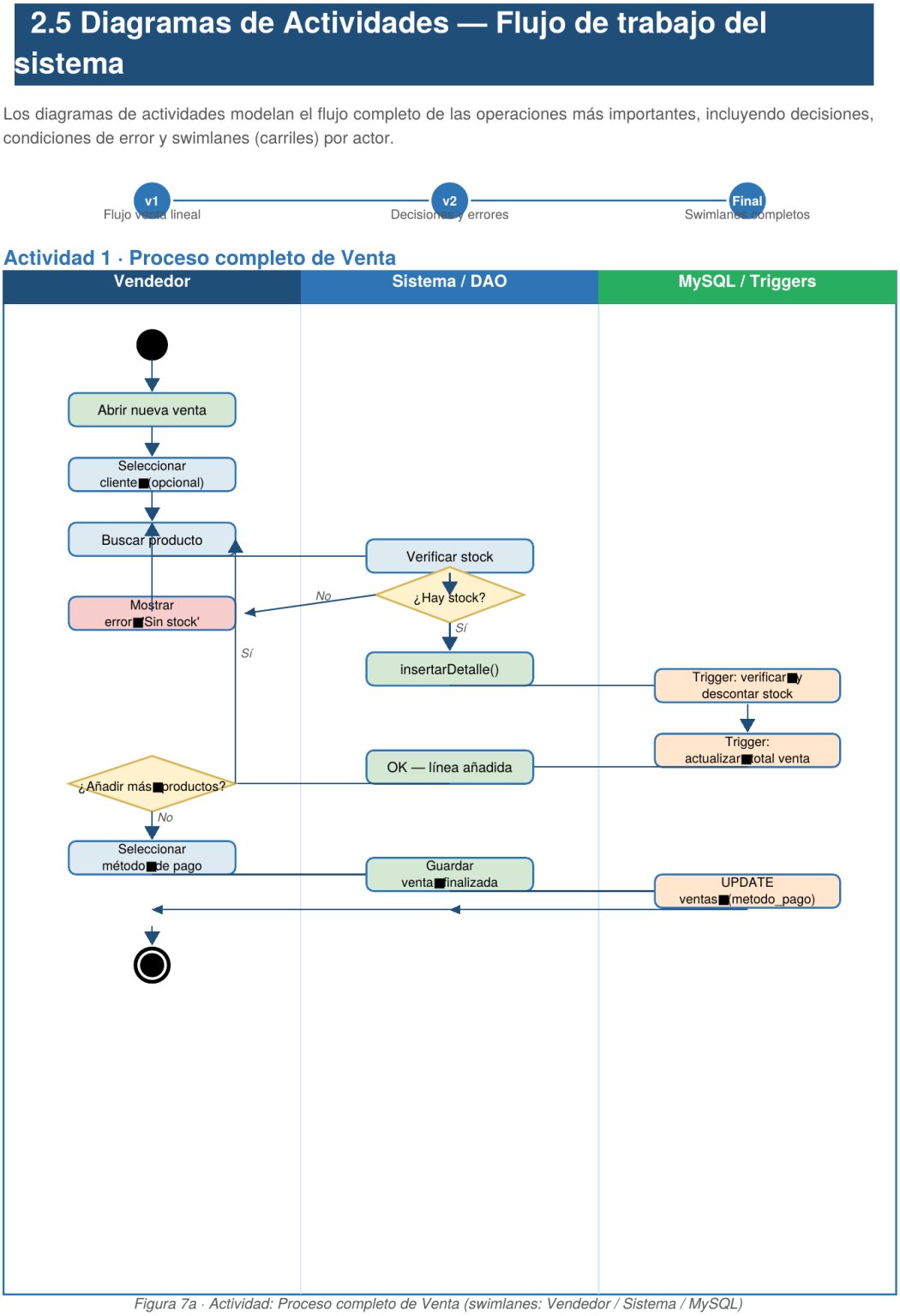

Figura 7a · Actividad: Proceso completo de Venta (swimlanes: Vendedor / Sistema / MySQL)

Figura 7a · Actividad: Proceso completo de Venta

Actividad 2 · Registrar nuevo Cliente

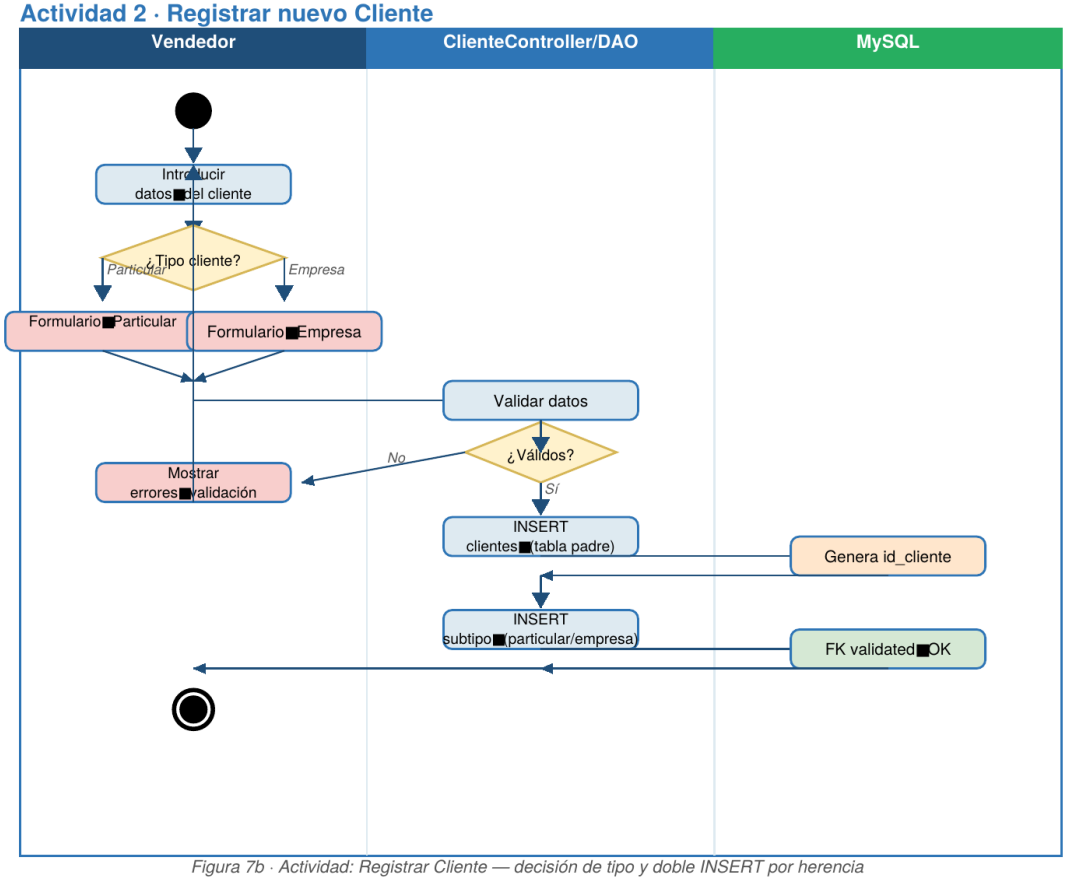

Figura 7b · Actividad: Registrar Cliente — decisión de tipo y doble INSERT por herencia

Actividad 3 · Login y control de acceso por rol

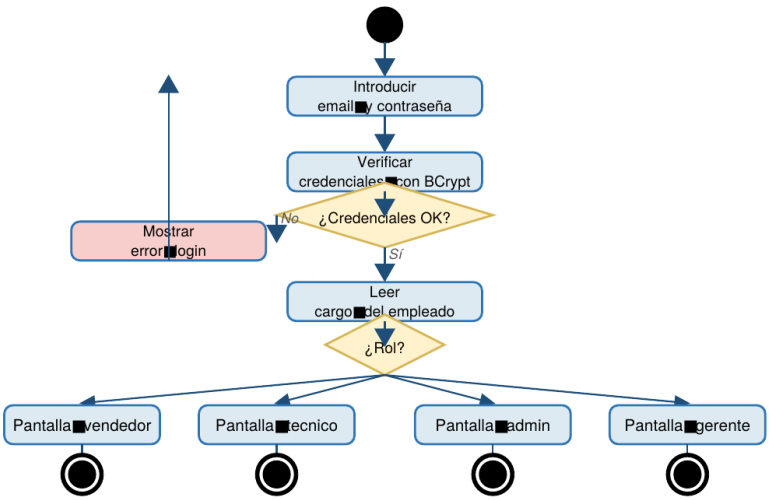

Figura 7b/7c · Actividad: Registrar Cliente y Login por rol

Resumen del proyecto

Artefacto	Estado	Herramienta
Diagrama ER	✓ Entregado	draw.io
Diagrama Relacional	✓ Entregado	MySQL Workbench
SQL creación BD	✓ Entregado	MySQL 8
Datos iniciales SQL	✓ Entregado	MySQL 8
Diagrama de Clases UML	✓ Entregado	draw.io
Casos de Uso	✓ Entregado	draw.io
Diagrama de Actividades	✓ Entregado	draw.io
Diagrama de Comportamiento/Secuencia	✓ Entregado	draw.io